

Informe Técnico: API RESTful de Gestión de Reservas

Estudiante: Jose Alejandro Terrazas Antezana **Materia:** Tecnología Web **Entrega:** Segunda Entrega (EC 3)

1. Introducción del Proyecto

El objetivo principal de esta aplicación Backend es gestionar el sistema de reservas de almuerzos de un restaurante. El enfoque principal es desarrollar una API RESTful robusta que permita crear, consultar, modificar y cancelar reservas, aplicando una arquitectura por capas (Dominio, Infraestructura, API), validaciones estrictas y patrones de diseño de software para la persistencia de datos.

La implementación utiliza:

- **ASP.NET Core** para la capa de API.
- **Dapper** (Micro-ORM) para optimizar las consultas de lectura (GETs).
- **Patrón Repositorio y Unidad de Trabajo (Unit of Work)** para encapsular la lógica de acceso a datos.
- **Inyección de Dependencias** para desacoplar los componentes.
- **Manejo Global de Excepciones y Excepciones Personalizadas** para la robustez.
- **AutoMapper** para la transformación de DTOs.
- **FluentValidation** para la validación de DTOs de entrada.
- **Swagger/OpenAPI** para la documentación interactiva.

2. Casos de Uso Descritos

Se han identificado las siguientes funcionalidades principales (Casos de Uso):

Prioridad Alta (CRUD básico y validaciones fundamentales)

- **UC.1: Crear Reserva (InsertReserva)**
 - **Descripción:** Permite registrar una nueva reserva de almuerzo.
- **UC.2: Obtener todas las Reservas (GetAllReservas)**
 - **Descripción:** Permite listar todas las reservas registradas en el sistema.
- **UC.3: Obtener Reserva por ID (GetReservaById)**
 - **Descripción:** Permite obtener los detalles de una reserva específica.

Prioridad Media

- **UC.4: Modificar Reserva (UpdateReserva)**
 - **Descripción:** Permite actualizar campos de una reserva existente.
- **UC.5: Cancelar Reserva (CancelReserva)**

- **Descripción:** Permite cambiar el estado de una reserva a "Cancelada".
- **UC.6: Obtener Detalles de Pedido de una Reserva (GetDetallesPedido)**
 - **Descripción:** Permite obtener la lista de platos (`DetallePedido`) asociados a una reserva.
- **UC.7: Registrar Detalle de Pedido a Reserva (AddDetallePedido)**
 - **Descripción:** Permite asociar una pre-orden de platos a una reserva.
- **UC.8: Gestionar Clientes (CRUD)**
 - **Descripción:** Permite la creación, lectura, actualización y eliminación de Clientes.
- **UC.9: Gestionar Horarios (CRUD)**
 - **Descripción:** Permite la creación, lectura, actualización y eliminación de Horarios (turnos).

3. Criterios de Aceptación (Reglas de Negocio)

Los siguientes criterios definen las reglas de negocio que la API debe validar:

Integridad de la Reserva

- Una Reserva pertenece a un único Cliente y un único Horario.
- No puede existir una reserva sin un `ClienteId` y `HorarioId` válidos.

Unicidad del Cliente

- El campo `Email` en la tabla `Clientes` debe ser único.

Estado Inicial

- El estado inicial de cualquier reserva siempre debe ser "Pendiente" (o "Confirmada" si la validación de disponibilidad es exitosa).

Validación Temporal

- `FechaReserva` y `HoraSolicitada` deben referirse a un momento en el futuro (o al menos a la fecha actual).
- La hora solicitada debe estar dentro del rango (`HoraInicio`, `HoraFin`) del `HorarioId` seleccionado.

Restricciones de Cantidad (CHECK)

- `Reserva.NumeroPersonas` debe ser mayor a 0.
- `DetallePedido.Cantidad` debe ser mayor a 0.

Disponibilidad (Criterio Crítico)

- El sistema solo debe permitir la creación/modificación de reservas si la suma total de `NumeroPersonas` (de reservas activas) no excede la `CapacidadMaxima` del `Horario` para esa fecha específica.

Restricción de Pago

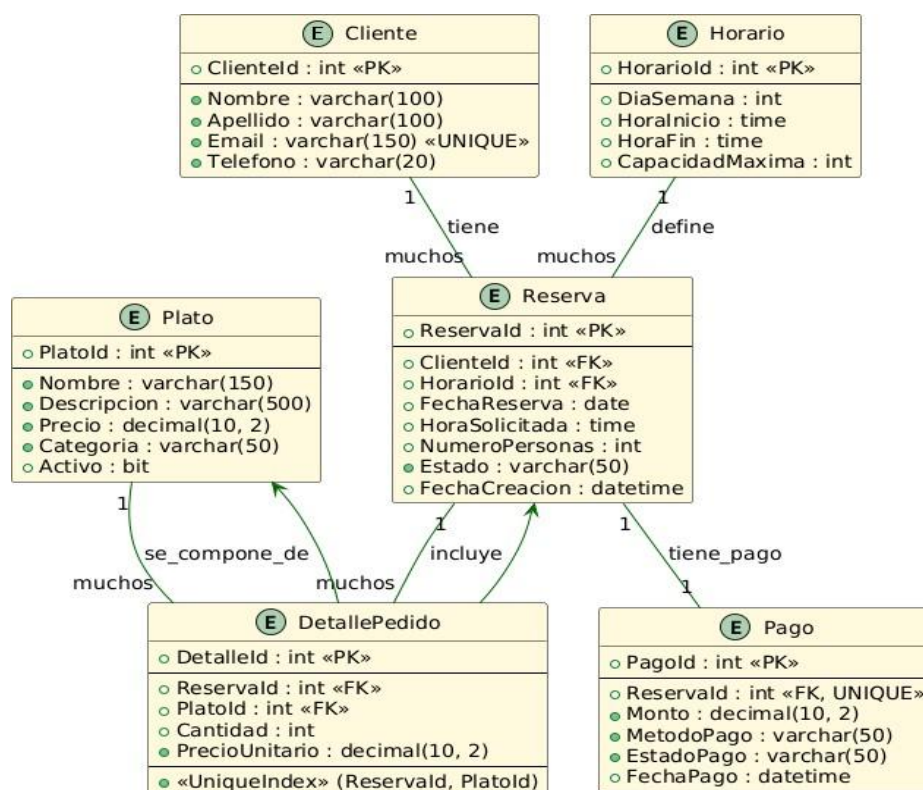
- Solo puede haber un pago por reserva (restricción `UNIQUE` en `Pago.ReservaId`).

Actualización Restringida

- No se puede modificar (PUT) ni registrar detalles de pedido (POST/Detalle) en reservas que ya han sido "Canceladas" o "Completadas".

4. Diagrama de Entidad-Relación (DER) de la Base de Datos

El diseño de la base de datos se ha modelado para reflejar las entidades y relaciones descritas.



5. Caso de Uso Trabajado (Evaluación EC 3)

Para la presente etapa de evaluación (EC 3), el trabajo se ha centrado en la implementación robusta de la arquitectura de la API y la aplicación de los patrones y tecnologías requeridos, materializados en los siguientes casos de uso:

Caso de Uso Principal Implementado:

- **UC.2: Obtener todas las Reservas (GetAllReservas)**
 - Este *endpoint* (`GET /api/reservas`) fue el foco principal, ya que integra la mayoría de los requisitos de la EC 3.
 - **Aplicación de Dapper:** La consulta se realiza utilizando Dapper (a través de `QueryMultipleAsync`) para optimizar la lectura, incluyendo *joins* con las tablas `Clientes` y `Horarios` y el conteo total en una sola llamada a la base de datos.
 - **Paginación:** Se implementó la paginación completa (`PagedList`) y la recepción de parámetros de consulta (`ReservaQueryFilter`) para `PageNumber` y `PageSize`.
 - **Estandarización JSON:** La salida del *endpoint* se envuelve en un objeto `ApiResponse<T>` que incluye la `Data` (los DTOs), un `Message` y la `PaginationMetadata`.

Casos de Uso Secundarios Implementados:

- **UC.8 y UC.9 (CRUD de Clientes y Horarios):** Se implementaron los controladores, servicios, repositorios y DTOs para la gestión completa de Clientes y Horarios.
- **UC.1 (Crear Reserva):** Se implementó la lógica de negocio en el servicio (`ReservaService.cs`) que valida la capacidad disponible (`CapacidadInsuficienteException`) antes de permitir la creación de una reserva.

Aspectos Transversales Implementados (EC 3):

- **Patrón Repositorio y Unit of Work:** Se implementó `IUnitOfWork` para coordinar los repositorios (`ReservaRepository`, `ClienteRepository`, etc.).
- **Excepciones:** Se crearon `BusinessException` y `CapacidadInsuficienteException` (Excepciones Personalizadas) y se configuró el `GlobalExceptionHandler` para capturarlas y devolver errores 400 `Bad Request` estandarizados.
- **Documentación con Swagger:** Se configuró `Program.cs` para leer comentarios XML y se documentó el `ReservasController.cs` (y los nuevos controladores) con `/// <summary>`, `/// <param>` y `/// <response>`.

Caso de Uso Trabajado (Evaluación EC 4)

Para esta etapa de evaluación (EC 4), el trabajo se centró en implementar la **Seguridad de la API** mediante **JSON Web Tokens (JWT)** y mecanismos robustos de autenticación para asegurar las funcionalidades de gestión de reservas, clientes y horarios ya desarrolladas.

Caso de Uso Principal Implementado (Transversal):

Funcionalidad	Descripción
UC.10: Autenticación y Gestión de Usuarios (Login y Register)	Permite que los usuarios se registren de forma segura y obtengan un token de acceso (JWT) para interactuar con los <i>endpoints</i> protegidos de la API.

Implementación Crítica de la Seguridad (EC 4):

- **Registro de Usuarios (Register):** Implementación de la lógica para crear nuevas credenciales de usuario.
- **Inicio de Sesión (Login):** Desarrollo del *endpoint* que valida las credenciales y genera un **JWT** que incluye los datos de autenticación del usuario.
- **Hashing de Contraseñas:** Se adoptó el estándar **PBKDF2** mediante la implementación del servicio `IPasswordService` para asegurar que las contraseñas se almacenen de forma segura y encriptada en la base de datos.
- **Configuración JWT (Autenticación):** Se configuró el esquema de autenticación `JwtBearer` en `Program.cs`, incluyendo la validación de Emisor (`Issuer`), Audiencia (`Audience`), y la firma del token (`SecretKey`), asegurando que solo los tokens válidos puedan acceder a los recursos protegidos.
- **Versionamiento (v1.0):** Se formalizó la configuración de versionamiento de la API, permitiendo futuras evoluciones (como v2.0) sin romper la compatibilidad con las aplicaciones existentes.
- **Documentación de Seguridad en Swagger:** Se ajustó la documentación de Swagger para incluir el botón de "Authorize", permitiendo a los usuarios ingresar su JWT y probar los *endpoints* protegidos directamente desde la interfaz.